

Human-in-the-Loop UX — Khi Nào Agent Cần Xin Phép?

AICB-P2T3 · Ngày 27 · Chương 6 — Agent trong Production

Giảng viên - Ngô Thanh Tùng

VinUniversity · Phase 2 · Track 3 · Tuần 6

HÃY SUY NGHĨ...

"Agent tự quyết hay hỏi người dùng — ranh giới nào là an toàn và không làm phiền?"



Giữ câu hỏi này trong đầu suốt buổi học hôm nay

Nội dung vận hành

1. Tại sao Full Autonomy nguy hiểm?
2. HITL Taxonomy — 5 Interaction Patterns
3. Confidence Routing — Khi nào interrupt?
4. Approval Workflows & Implementation
5. Feedback Loops & Audit Trails
6. HITL UX Best Practices

HUMAN-IN-THE-LOOP (HITL) TRONG AI ENGINEERING

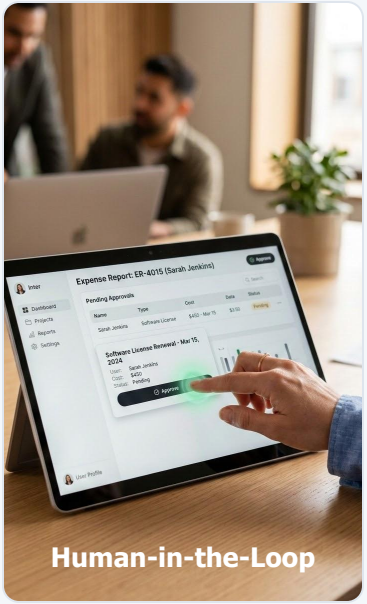
Ranh giới giữa Tự trị hoàn toàn và Kiểm soát an toàn

Định hướng & Mục tiêu

Nội dung: Thiết kế hệ thống Agent an toàn & Tối ưu hóa điểm chạm con người.

Mục tiêu: Hiểu rõ giới hạn của AI, phân loại rủi ro, và xây dựng quy trình duyệt (approval workflows) không gây quá tải cho người vận hành.





VINUNIVERSITY

The Illusion of Full Autonomy & The Need for HITL

1

Establish the "Why" through high-stakes real-world failures and risk modeling.

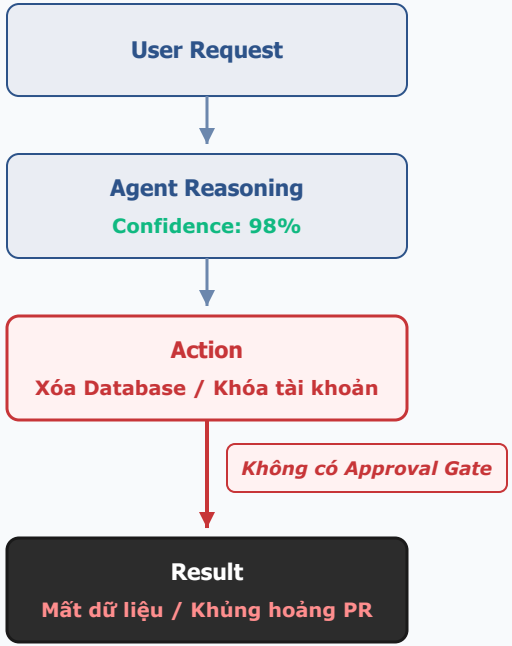
The Hook - Agent tự quyết, chuyện gì xảy ra?

Sức hấp dẫn và Cạm bẫy của Full Autonomy

- **Kỳ vọng:** Agent nhận request Tự suy luận (Reasoning) Tự hành động (Action) Tiết kiệm 100% thời gian.
- **Thực tế:** Thiếu bối cảnh kinh doanh (Business Context) + Ảo giác (Hallucination) = Thảm họa.

Agent không cảm nhận được hậu quả của việc mất khách hàng hay bị phạt pháp lý.

Mô phỏng Failure Path



Tại sao "Thông minh" không đồng nghĩa với "An toàn"?

VINUNIVERSITY

Độ tự tin của LLM (LLM Confidence) ≠ Độ chính xác thực tế

Vấn đề Calibration

Các mô hình ngôn ngữ lớn thường rất "tự tin" ngay cả khi chúng sai (Overconfidence).

Ngữ cảnh hẹp (Narrow Context)

Agent chỉ nhìn thấy số liệu trong prompt, không thấy được chiến lược tổng thể của ngân hàng hay mối quan hệ với khách hàng VIP.

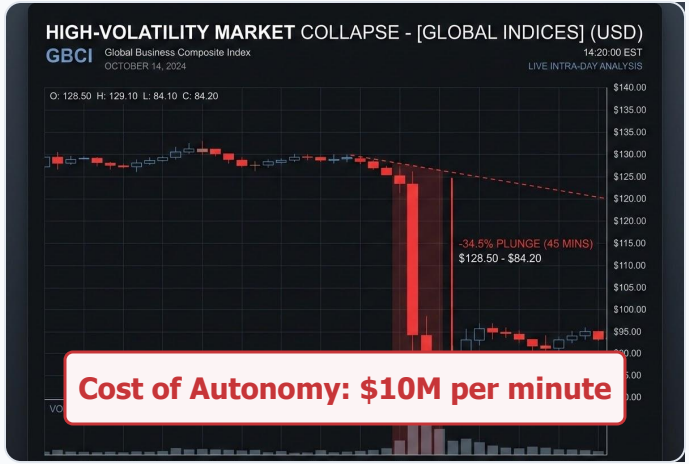
Kết luận: Đừng bao giờ giao phó (delegate) rủi ro kinh doanh cho một con số xác suất do AI tạo ra mà không có màng lọc con người.

Sự cố thực tế: Knight Capital Group (2012)

Khi tốc độ hủy diệt doanh nghiệp

- **Sự kiện:** Thuật toán trading tự động gặp lỗi do cấu hình sai.
- **Hành động:** Agent liên tục mua cao, bán thấp hàng triệu cổ phiếu trên thị trường thật.
- **Hậu quả:** Bốc hơi **bốc hơi $440 triệu USD chỉ trong 45 phút**.
- **Nguyên nhân cốt lõi:** Không có "Kill Switch" thủ công, không có Human-in-the-Loop khi volume giao dịch vượt ngưỡng bất thường.

Visualizing the 45-Minute Plunge



Reference: SEC's investigative report on Knight Capital Group's 2012 trading software glitch.

Sự cố thực tế: Air Canada Chatbot (2024)

Khi AI tự viết ra chính sách

- **Sự kiện:** Chatbot tự động tư vấn chính sách hoàn tiền vé máy bay sai lệch cho khách hàng.
- **Hành động:** Khách hàng làm theo, Air Canada từ chối hoàn tiền với lý do "Chatbot tự nói, hãng không chịu trách nhiệm".
- **Phán quyết của Tòa án:** Doanh nghiệp **phải chịu trách nhiệm** cho mọi phát ngôn và hành động của Agent. Hãng thua kiện.
- **Bài học:** Full Autonomy cho Customer-facing agents (Agent giao tiếp trực tiếp với khách) mang rủi ro pháp lý khổng lồ.

Court Ruling & Chatbot Liability



Reference: Moffatt v. Air Canada (2024) - Civil Resolution Tribunal.

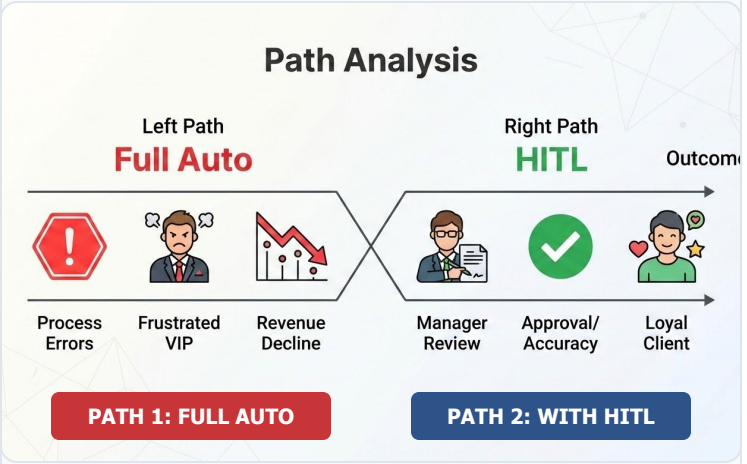
Rủi ro "Black Box" trong Phân tích Dữ liệu Nội bộ

Tác động ngầm: Khi Agent ra quyết định sai về Khách hàng

Mất đi nguồn thu nhập trọn đời từ khách hàng VIP

- **Kịch bản:** Agent được cấp quyền tự động khóa thẻ tín dụng hoặc hạ hạn mức tín dụng dựa trên một mô hình phân tích rủi ro.
- **Lỗi hệ thống:** Agent nhận định sai dấu hiệu giao dịch bất thường ở một khách hàng Priority (VIP).
- **Hậu quả:**
  - Trải nghiệm khách hàng bị phá vỡ hoàn toàn.
  - Mất đi nguồn thu nhập trọn đời (**Total Operating Income - TOI**) từ khách hàng cao cấp.
- **Giải pháp:** Mọi hành động tác động trực tiếp đến vòng đời khách hàng (CLV) phải đi qua trạm kiểm duyệt.

Decision Paths: Full Autonomy vs. HITL



Path 1 (Full Auto): *Agent blocks VIP card instantly based on false positive risk score -> High churn risk & massive TOI loss.*

Path 2 (HITL Guardrail): *Agent flags anomaly -> Human manager rejects incorrect flag -> Customer relationship & loyalty maintained.*

Khung đánh giá hành động (Action Assessment Framework)

VINUNIVERSITY

Reversible vs. Irreversible Actions

Không phải mọi hành động đều bình đẳng

Hành động có thể đảo ngược (Reversible)

Chi phí sai lầm thấp:

- Draft một email phản hồi
- Truy vấn dữ liệu (**SELECT**)
- Chạy mô hình giả lập (Simulation)
- Đề xuất danh sách khách hàng rời bỏ

Hành động không thể đảo ngược (Irreversible)

Chi phí sai lầm cao:

- Gửi email thẳng cho Khách hàng/Báo chí
- Chỉnh sửa/Xóa dữ liệu (**UPDATE, DROP**)
- Đưa mô hình lên Production
- Tự động hủy tài khoản của khách hàng

Nguyên tắc: Full Autonomy chỉ dành cho cột bên trái!

Ma trận Tác động Kinh doanh (Business Impact Matrix)

Định lượng mức độ nguy hiểm của Tool



Nguyên tắc: Các hành động có tác động cao hoặc khó đảo ngược phải được kiểm soát nghiêm ngặt!

Cost of Error (Chi phí của sự sai lầm)

VINUNIVERSITY

Tại sao không để Agent tự dự đoán và ra quyết định?

Hãy xem xét kịch bản dự đoán khách hàng rời bỏ (Churn Prediction) kết hợp tính toán Total Operating Income (TOI).

Kịch bản & Phân tích

Agent Output:

Dự đoán khách hàng A sẽ churn, đề xuất tung gói ưu đãi giảm lãi suất thẻ tín dụng để giữ chân.

False Positive (Dự đoán sai):

Khách hàng A không hề có ý định rời đi. Việc tự động cấp ưu đãi khiến ngân hàng mất một khoản TOI khổng lồ trong 2 năm tiếp theo một cách vô ích.

Cost of Error (Hậu quả)

Bản chất của thiệt hại:

Bằng chính số tiền biên lợi nhuận bị mất do Agent "quá tay".

Khác biệt với phần mềm truyền thống:

Khác với phần mềm thông thường (lỗi thì crash), Agent lỗi sẽ làm **chảy máu tài chính** một cách thầm lặng.

Cost of Error vs. Cost of Interruption

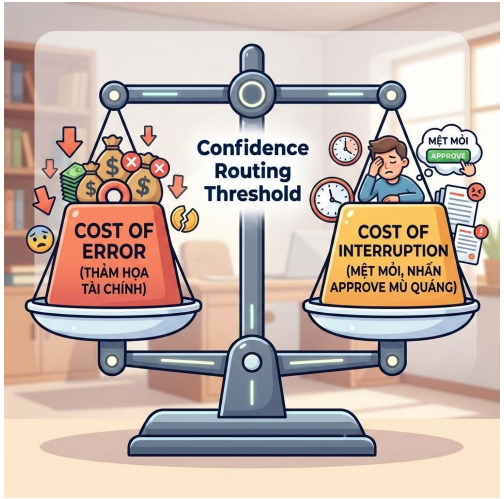
Bài toán Cân bằng: An toàn hay Năng suất?

Cost of Error

Rủi ro từ Agent tự quyết:

Đã phân tích ở Slide trước bao gồm:

- Rủi ro tài chính
- Mất mát dữ liệu nghiêm trọng
- Ảnh hưởng uy tín hệ thống



Cost of Interruption

Chi phí gián đoạn:

- Thời gian của con người rất đắt đỏ.
- Nếu Agent hỏi ý kiến cho mọi giao dịch $10, nhân sự sẽ bị **"Alert Fatigue"** (Hội chứng mệt mỏi vì cảnh báo).
- **Hậu quả:** Nhân sự sẽ click "Approve" một cách vô thức như robot.

Nhiệm vụ của AI Engineer: Tìm ra "Sweet Spot" – Điểm cân bằng hoàn hảo

Sử dụng các thuật toán **Confidence Routing** (Định tuyến theo độ tự tin) để chỉ ngắt quãng con người khi thực sự cần thiết, tối ưu hóa giữa an toàn vận hành và hiệu suất lao động.

VINUNIVERSITY

HITL Taxonomy & Interaction Patterns

Categorize how humans and AI interact.

2

The Autonomy Spectrum - Phổ Tự trị của AI

Từ "Chạy bằng cơm" đến "Tự động hóa hoàn toàn"

Không có một thiết kế "one-size-fits-all" (một cho tất cả). Mức độ tự trị của Agent nằm trên một dải phổ:



Sweet Spot
Vị trí lý tưởng cho Enterprise AI

Full Manual	HITL Strict	HITL Balanced	Full Auto
Hỗ trợ tra cứu	**Kiểm soát nghiêm ngặt**	**Kiểm duyệt tối ưu**	**Tự động hoàn toàn**
Con người làm mọi thứ từ đầu đến cuối, công cụ chỉ hỗ trợ tìm kiếm và tra cứu thông tin nền tảng.	Agent chỉ đề xuất phương án, con người bắt buộc phải duyệt duyệt qua từng hành động trước khi thực thi.	Trạm kiểm duyệt chỉ kích hoạt linh hoạt dựa trên đánh giá mức độ rủi ro hoặc độ tự tin của quyết định.	Agent hoàn toàn tự quyết và thực thi hành động trực tiếp mà không cần bất kỳ sự can thiệp nào từ con người.

An toàn tối đa - Tốc độ thấp

Tốc độ tối đa - Rủi ro cao

Phân biệt "Human-in-the-Loop" và "Human-on-the-Loop"

VINUNIVERSITY

Gatekeeper (Người gác cổng) vs. Supervisor (Người giám sát)

Human-in-the-Loop (HITL)

- **Cơ chế:** Active Blocking. Hành động bị "đóng băng" cho đến khi có phê duyệt.
- **Use case:** Chuyển tiền, thay đổi database, gửi email cho khách hàng.

Human-on-the-Loop (HOTL)

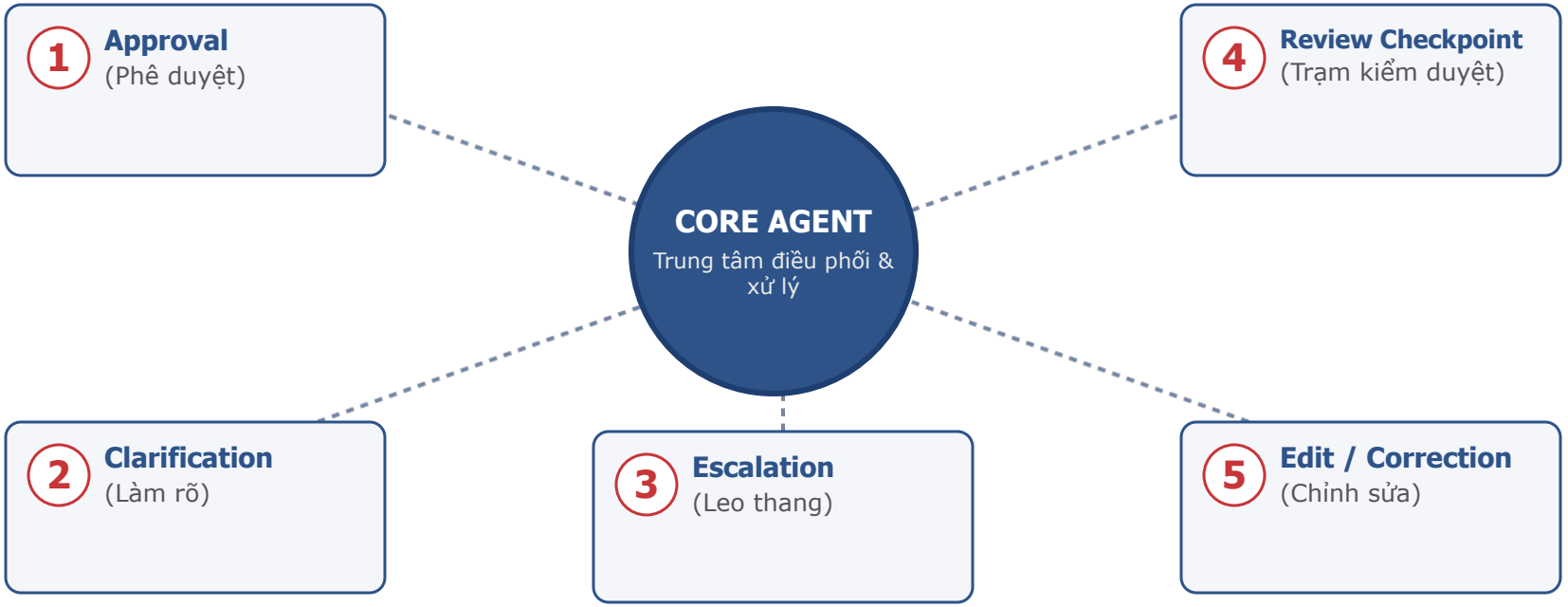
- **Cơ chế:** Passive Monitoring. Agent tự động thực hiện hành động, nhưng con người có thể theo dõi real-time và can thiệp (Kill switch/Rollback) nếu thấy sai.
- **Use case:** Gửi thông báo nội bộ, query dữ liệu, phân loại ticket hỗ trợ.

5 Mẫu Tương tác Cốt lõi (Interaction Patterns)

Taxonomy: Cách Con người và AI "Nói chuyện" với nhau

Khi thiết kế hệ thống, chúng ta sử dụng 5 pattern chính để nhúng con người vào quy trình:

**Lựa chọn pattern phụ thuộc vào bài toán cụ thể và tính chất của dữ liệu đầu vào/đầu ra.*



Pattern 1: Approval (Kẻ gác đền)

Ngăn chặn các hành động Destructive hoặc High-Value

Đặc điểm:

Flow thực thi dừng lại hoàn toàn. Yêu cầu một tín hiệu nhị phân: Approve hoặc Reject.

Ví dụ trong Banking:

- Agent tự động phát hiện gian lận và đề xuất khóa tài khoản → Chờ Manager duyệt.
- Lệnh chuyển tiền (Wire transfer) > $10,000 sinh ra từ hệ thống tự động.
- Hành động mang tính phá hủy (Destructive): DROP TABLE, DELETE user_records.

Pattern 2: Clarification (Hỏi lại cho chắc)

Xử lý Input mơ hồ từ người dùng

Đặc điểm:

- Xảy ra ở giai đoạn đầu của flow (Pre-execution).
- Agent nhận ra nó thiếu thông tin để ra quyết định an toàn.

Quy trình:

Nhận Input → Nhận diện mơ hồ (Ambiguity Detection) → Đặt câu hỏi → Chờ Input bổ sung.

Ví dụ thực tế & Ý nghĩa

User: "Cho tôi xem báo cáo doanh thu Q1."

Agent: "Bạn muốn xem báo cáo doanh thu Q1 của khối Bán lẻ (Retail) hay Doanh nghiệp (Corporate)?"

Tác dụng của Clarification:

Ngăn chặn Agent thực thi các truy vấn nặng (heavy queries) trên sai tập dữ liệu, giúp tiết kiệm tài nguyên tính toán và thời gian của hệ thống.

Pattern 3: Escalation (Chuyển cho chuyên gia)

Khi Agent biết giới hạn của mình (Out-of-Domain)

Đặc điểm: Flow thực thi dừng lại hoàn toàn. Agent nhường quyền kiểm soát (handoff) hoàn toàn cho con người khi gặp ca khó hoặc ngoài phạm vi được phép.

Khi nào kích hoạt?

- Độ tự tin (Confidence) thấp.
- Phát hiện các từ khóa nhạy cảm (Compliance, Legal, Kiện tụng).

Ví dụ:

- Chatbot CSKH đang xử lý một case thu hồi nợ, khách hàng dọa kiện → Agent lập tức dừng gen text và route ticket sang bộ phận Pháp lý.

Pattern 4: Review Checkpoint (Trạm kiểm duyệt kết quả)

Xác minh output phức tạp trước khi phát hành

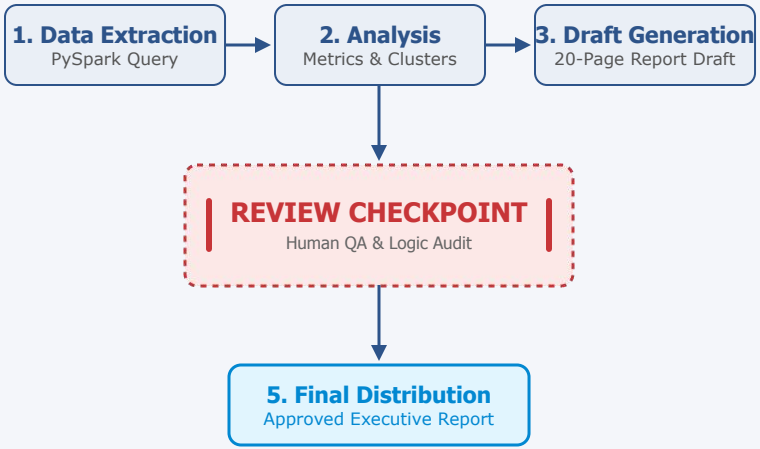
Đặc điểm:

- Agent hoàn thành toàn bộ khối lượng công việc nặng (Heavy lifting), tạo ra một bản nháp (Draft).
- Con người chỉ đóng vai trò Quality Assurance (QA).

Ví dụ: Báo cáo Credit Card TOI

- Agent query hàng triệu dòng giao dịch bằng PySpark, tính toán metrics phức tạp, phân cụm khách hàng, sinh báo cáo 20 trang.
- Báo cáo dừng lại ở Checkpoint để Data Scientist review logic và số liệu, không tự động gửi email cho Ban Giám đốc.

Mô hình Quy trình (Pipeline Flow)



** Toàn bộ quy trình nặng được tự động hóa, con người chỉ giữ chốt kiểm duyệt cuối.*

Pattern 5: Edit / Correction (Chỉnh sửa nội dung)

Khi "Approve/Reject" là không đủ

Đặc điểm: Thay vì chỉ từ chối bản nháp (khiến Agent phải chạy lại từ đầu tốn tiền API), con người trực tiếp nhảy vào sửa đổi nội dung.

Sức mạnh cộng gộp: Bản chỉnh sửa (Delta) của con người chính là dữ liệu vàng (Golden data) để fine-tune hoặc thêm vào bộ nhớ (Few-shot learning) của Agent sau này.

Ví dụ:

- Agent gen ra đoạn code Pandas để dự đoán khách hàng rời bỏ (Churn prediction).
- Con người sửa lại vài tham số LightGBM cho chuẩn, sau đó bấm chạy tiếp.

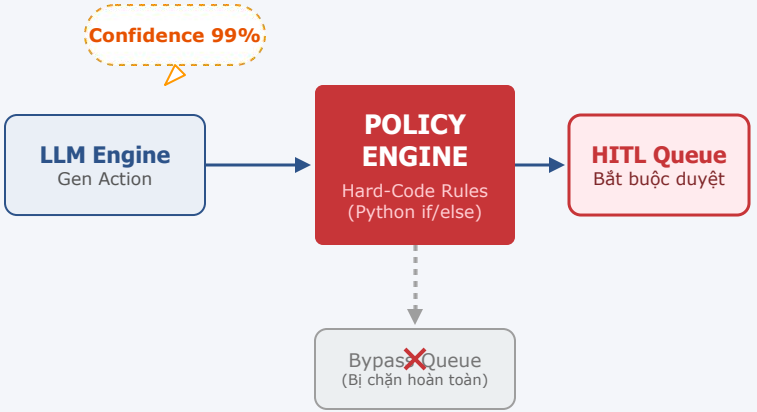
Risk-Based Routing Policy (Hard Rules)

Khi thuật toán nhường bước cho pháp lý

Nguyên tắc & Kiến trúc

- **Luật bất thành văn:** Dù thuật toán (LLM) trả về độ tự tin tự đánh giá (Confidence Score) là 99.99%, một số hành động vẫn BẮT BUỘC phải đi qua cửa con người.
- **Kiến trúc:** Policy rules phải được hard-code ở tầng Application (ví dụ: dùng Python if/else), không được giao cho LLM tự quyết định có nên bypass hay không.

Mô hình hoạt động (Policy Bypass Block)



** Policy Engine hoạt động như một bức tường lửa, bỏ qua mọi mức độ tự tin (Confidence) của mô hình khi vi phạm Hard Rules.*

Vùng cấm địa của Full Autonomy (Red Zones)

Dưới đây là các tác vụ luôn yêu cầu HITL Strict hoặc Escalation:

1. **Chỉnh sửa dữ liệu PII (Personally Identifiable Information):** Thay đổi CCCD, số điện thoại, địa chỉ nhà của khách hàng.
2. **Giao dịch tài chính (Financial Execution):** Khởi tạo lệnh chuyển tiền, duyệt khoản vay, cấp hạn mức tín dụng.
3. **Tương tác hệ thống lõi (Core Banking):** Mọi lệnh can thiệp (Write/Update) vào dữ liệu gốc (Golden Source).
4. **Bỏ qua cảnh báo tuân thủ (Overriding Compliance Flags):** Agent không được tự ý gỡ cờ "Nghi ngờ rửa tiền (AML)" của bất kỳ tài khoản nào.

VINUNIVERSITY

Confidence Routing & AI Self-Evaluation

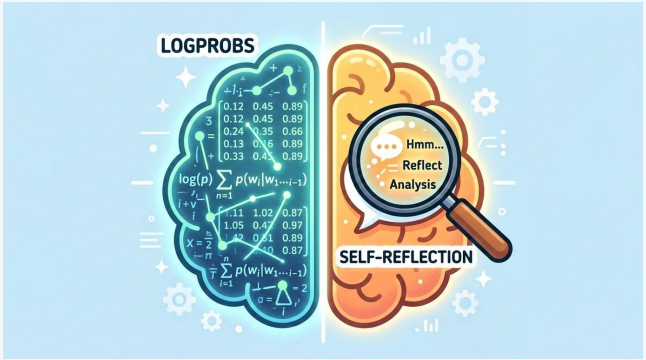
3

How does the agent actually know it needs help?

Agent làm sao biết mình cần trợ giúp?

Để xây dựng cơ chế định tuyến (Routing), Agent cần khả năng tự định lượng sự không chắc chắn (Uncertainty). Có hai phương pháp chính để trích xuất độ tự tin từ một LLM:

- **Log Probabilities (Logprobs):** Đánh giá dựa trên xác suất token nội tại của mô hình (Mức độ Machine-level).
- **Prompt-based Self-Reflection:** Yêu cầu mô hình tự suy ngẫm và chấm điểm (Mức độ Semantic-level).



Cấu trúc tự đánh giá

Logprobs (Xác suất) & Self-Reflection (Tự suy ngẫm)

Log Probabilities (Logprobs)

VINUNIVERSITY

Đo lường độ tự tin qua Toán học (Token Probability)

Bản chất & Cách hoạt động

- **Bản chất:** LLM dự đoán từ tiếp theo. API (như OpenAI) có thể trả về xác suất của từng token được sinh ra.
- **Cách hoạt động:** Tính trung bình (Average) xác suất của chuỗi token quyết định (ví dụ: chuỗi JSON gọi tool).
- Xác suất > 95% → Mô hình rất chắc chắn.
- Phân phối xác suất phẳng (Flat distribution) giữa nhiều lựa chọn → Mô hình đang phân vân.

Ưu điểm & Nhược điểm

Ưu điểm:

- Khách quan, chi phí thấp (không tốn thêm token).

Nhược điểm:

- Khó mapping trực tiếp logic kinh doanh; một số LLM API đóng không cung cấp logprobs đầy đủ.

Prompt-based Self-Reflection

Đo lường độ tự tin qua Prompting (Tự suy ngẫm)

Bản chất: Thêm một yêu cầu (instruction) vào System Prompt bắt LLM tự đánh giá.

Cấu trúc Prompt: `"Dựa trên context được cung cấp, hãy xuất ra action kèm theo một trường confidence score (từ 0.0 đến 1.0) và reasoning (lý do tại sao bạn tự tin mức đó)."`

Ưu điểm & Nhược điểm:

- **Ưu điểm:** Dễ dàng tích hợp vào JSON schema/Pydantic model. Giải thích được (Explainable) → Con người đọc reasoning để quyết định duyệt nhanh hơn.
- **Nhược điểm:** Tốn thêm token tính toán; dễ bị ảo giác (LLM overconfidence).

Pydantic Model & JSON Output

```
class AgentResponse(BaseModel):
    action: str
    confidence_score: float
    reasoning: str
```

```
{
  "action": "refund_order",
  "confidence_score": 0.9,
  "reasoning": "User provided the
correct invoice and the item was
marked as returned in DB."
}
```

The Overconfidence Problem

VINUNIVERSITY

Khi Agent "Ngốc nhưng Tự tin"

Sycophancy (Tính xu nịnh)

Sự thật tuyệt đối: LLM có xu hướng đồng tình với người dùng hoặc giả định rằng thông tin chúng bịa ra (hallucination) là sự thật tuyệt đối.

Nghịch lý

Sai logic: LLM có thể trả về confidence_score = 0.99 cho một hàm delete_database hoàn toàn sai logic.

Hậu quả

Thảm họa: Nếu chỉ dựa vào điểm tự tin thô (raw score) do LLM tự báo cáo, hệ thống sẽ tự động vượt qua trạm kiểm duyệt (Approval Gate), gây ra thảm họa dữ liệu.

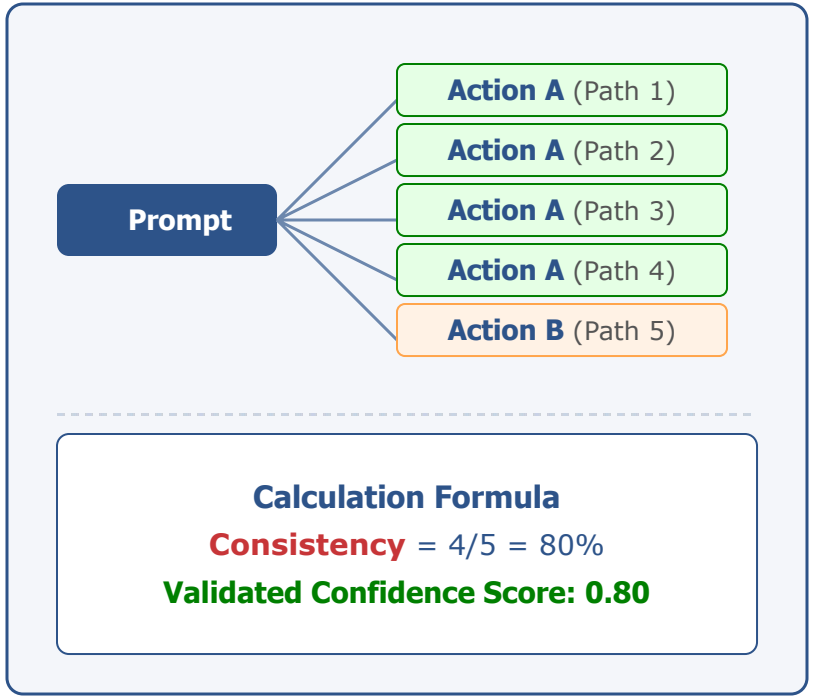
Kiểm tra tính nhất quán (Self-Consistency)

Vấn đề: Làm sao để ép LLM nói thật về độ tự tin?

Giải pháp: Sinh ra nhiều câu trả lời (Sampling với Temperature > 0.5) cho cùng một prompt.

Cơ chế:

- Chạy 5 luồng suy luận (reasoning paths) độc lập.
- Nếu 5 kết quả giống hệt nhau → Độ tự tin thực tế rất cao (High Calibrated Confidence).
- Nếu trả về 3 kết quả khác nhau → Độ tự tin thực tế thấp (Low Calibrated Confidence), cần gọi con người (Escalate).



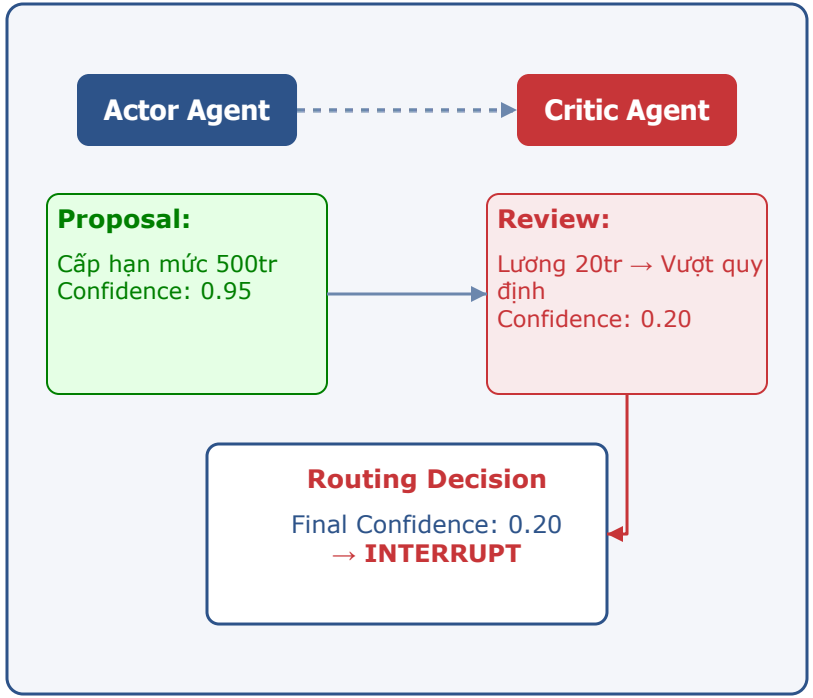
Tranh biện đa tác tử (Multi-Agent Debate)

Kiến trúc: Sử dụng một Agent thứ hai (Critic/Reviewer Agent) chuyên đánh giá lại quyết định của Agent thứ nhất (Actor Agent).

Flow:

1. **Actor:** "Tôi muốn cấp hạn mức thẻ 500 triệu. Confidence: 0.95."
2. **Critic:** "Kiểm tra rule: Lương khách hàng chỉ 20 triệu. Hạn mức vượt quá quy định. Chấm điểm Confidence: 0.20."
3. **Routing:** Dùng điểm của Critic làm mốc cuối cùng để quyết định Interrupt.

Kết quả: Giảm đáng kể tỷ lệ False Positive và loại bỏ sự "ảo tưởng" của Actor.



Luồng định tuyến (Confidence Routing Flow)

Phân luồng dựa trên điểm tự tin (Sau hiệu chuẩn)

Chúng ta chia mức độ tự tin thành 3 phân khu (Tiers) để tối ưu hóa UX và rủi ro:



High Confidence

Vùng an toàn tuyệt đối: Agent tự động thực thi.



Medium Confidence

Vùng trung gian: Agent đề xuất một bản nháp hoặc hành động, chờ con người xác nhận (One-click Approval).

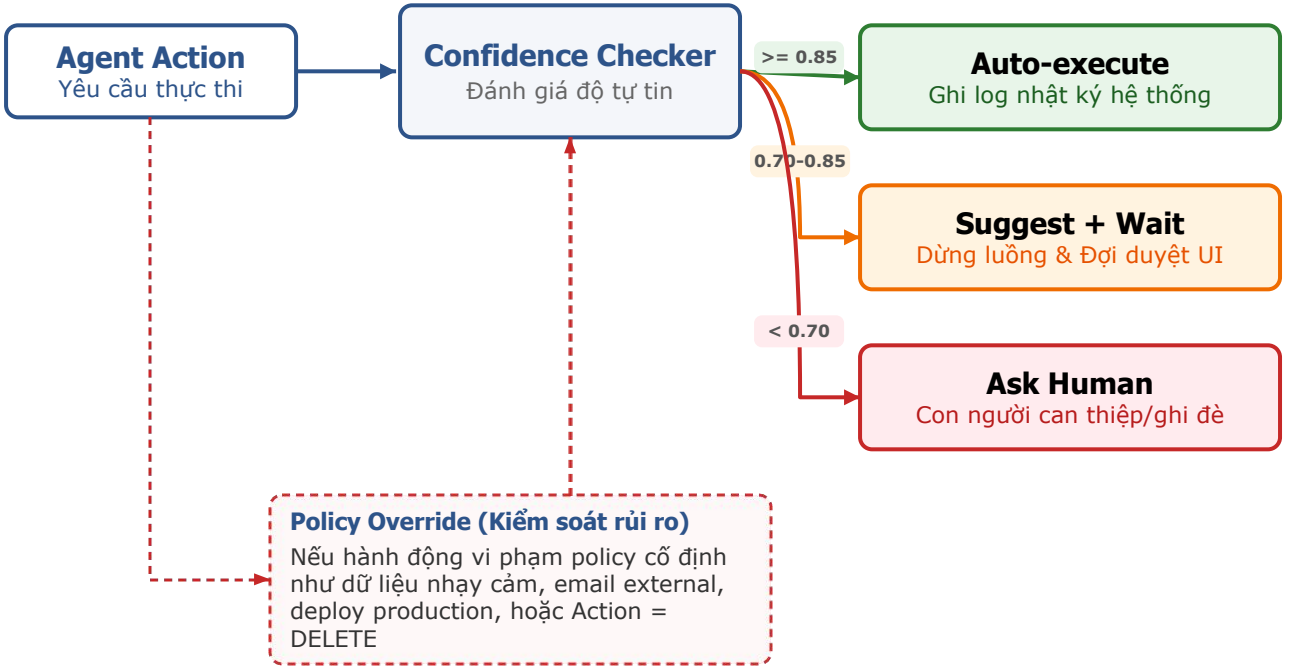


Low Confidence

Vùng mù mờ: Agent dừng lại hoàn toàn, đặt câu hỏi làm rõ (Clarification) hoặc đẩy toàn bộ context cho con người xử lý (Escalation).

LangGraph Routing Logic

Hệ thống phân luồng tự động dựa trên điểm tin cậy (Confidence Score) và các chính sách kiểm soát tối cao.



Empirical Tuning - Điểm Threshold không phải là phép màu

Không có "Magic Number" (Con số ma thuật) cho Confidence

Sai lầm phổ biến

Đặt cứng ngưỡng (threshold) ở mức cố định cho mọi dự án, ví dụ như luôn mặc định chọn con số:

0.85

Việc áp dụng một tiêu chuẩn duy nhất dễ dẫn đến các quyết định sai lầm khi đặc thù mỗi bài toán hoàn toàn khác nhau.

Thực tế khác biệt

Ý nghĩa và giá trị của cùng một ngưỡng độ tin cậy thay đổi hoàn toàn tùy theo ngữ cảnh ứng dụng:

Phân loại email nội bộ
Yêu cầu độ chính xác ở mức vừa phải, chấp nhận rủi ro sai sót nhỏ.

Mô hình dự đoán CLV
Tính toán giá trị trọn đời của khách hàng VIP đòi hỏi độ chuẩn xác cực kỳ nghiêm ngặt.

Empirical Tuning

Tinh chỉnh thực nghiệm

Ngưỡng tối ưu hoàn toàn không có một công thức chung hay "magic number" cố định.

Nó phải được đánh giá, cân nhắc và quyết định chặt chẽ dựa trên:

- Dữ liệu lịch sử (Historical data).
- Sự chấp nhận rủi ro của doanh nghiệp (Risk appetite).

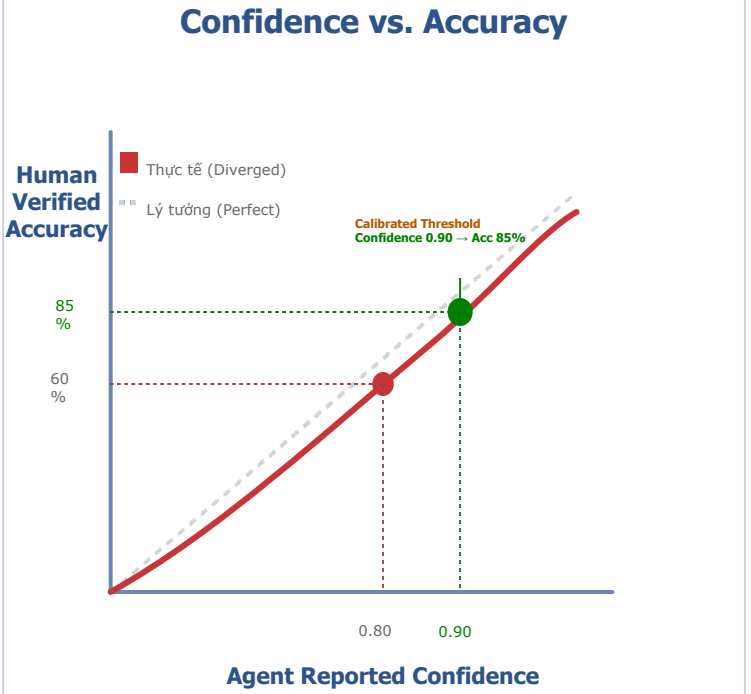
Hiệu chỉnh với Dữ liệu lịch sử (Calibrating with Data)

Phân tích Cost of Error để xác định Threshold

Bước 1: Shadow Mode (Chạy ngầm).
Cho Agent chạy song song với con người nhưng không có tác động thực tế (Chỉ sinh log).

Bước 2: Vẽ biểu đồ Precision/Recall vs. Confidence.
Khớp (Map) điểm tự tin do Agent tự đánh giá với độ chính xác thực tế được con người chấm điểm.

Bước 3: Tối ưu hóa (Optimization).
Nếu ngưỡng 0.80 dẫn đến accuracy thực tế chỉ 60% → Threshold quá lỏng, cần nâng lên 0.90.
Nếu Cost of Error rất đắt đỏ (ví dụ: $10K/lỗi) → Đẩy threshold lên mức cực đoan (0.98) hoặc bắt buộc duyệt (HITL Strict).



VINUNIVERSITY

Implementation with LangGraph

4

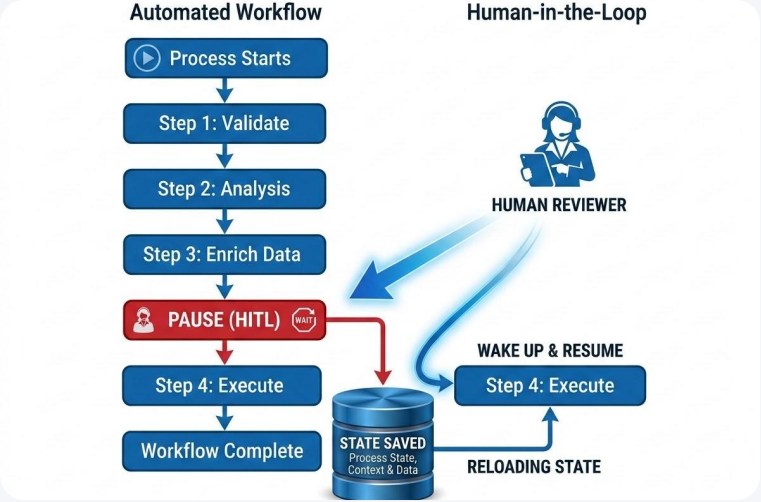
The technical core. How to actually build this in code.

State Management trong LangGraph

Tại sao HITL cần State Persistence (Lưu trữ trạng thái)?

- **Bản chất của Agent:** Các đoạn script Python thông thường chạy từ đầu đến cuối rồi kết thúc (Stateless).
- **Bài toán HITL:** Khi Agent dừng lại chờ con người duyệt (ví dụ chờ duyệt cấp ưu đãi cho khách hàng có rủi ro Churn), quá trình chờ này có thể mất 5 phút, 1 tiếng, hoặc vài ngày.
- **Giải pháp:** Đồ thị (Graph) phải có khả năng "ngủ đông" (Sleep/Suspend). Trạng thái hiện tại của đồ thị (các biến, lịch sử chat, quyết định của LLM) phải được lưu xuống đĩa hoặc database.
- **Công cụ:** LangGraph giải quyết bài toán này thông qua **Checkpointer**.

State Persistence Process Flow



LangGraph Checkpointer Workflow

Tự động đóng băng trạng thái khi cần Human-in-the-loop và khôi phục chính xác khi được kích hoạt lại.

Giới thiệu MemorySaver và Checkpointer

MemorySaver: Bộ nhớ ngắn hạn

- **In-memory mặc định:** Thích hợp cho quá trình dev và test đồ thị nhanh chóng.
- **Cơ chế tự động:** LangGraph chụp snapshot toàn bộ State ngay sau khi mỗi node thực thi.

Lưu ý quan trọng
Dữ liệu trong MemorySaver sẽ bị xóa hoàn toàn khi ứng dụng hoặc server restart.

Production Checkpointers

Trong môi trường doanh nghiệp và tài chính thực tế, hệ thống yêu cầu độ tin cậy tuyệt đối để tránh mất mát dữ liệu.

- **PostgresSaver:** Lưu trữ trạng thái bền vững vào cơ sở dữ liệu PostgreSQL.
- **RedisSaver:** Tối ưu hiệu năng đọc/ghi trạng thái với bộ nhớ đệm phân tán Redis.

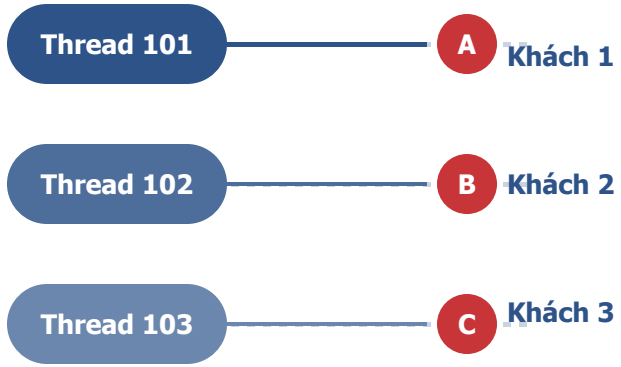
Thread ID - Quản lý nhiều phiên làm việc

Làm sao để Agent phục vụ 1000 khách hàng cùng lúc?

- **Khái niệm Thread ID:** Mỗi đồ thị đang chạy cần một ID định danh duy nhất (Ví dụ: customer_id_8472 hoặc ticket_id_991).
- **Cấu hình (Config):** Khi invoke graph, chúng ta truyền Thread ID qua tham số config.
- **Tác dụng:**
  - Giữ cho các luồng duyệt hoàn toàn độc lập.
  - Có thể truy cập chính xác trạng thái của Graph A mà không nhầm lẫn.

Cấu hình Thread ID:

```
config = {"configurable":
  {"thread_id": "churn_case_104"}
}
```



Chiến lược Ngắt (Interrupt Strategies)

Hai vị trí "đặt trạm gác" trong LangGraph

1. `interrupt_before`

Ngắt trước khi hành động

Dừng đồ thị hoàn toàn trước khi một node cụ thể được thực thi.

Trường hợp áp dụng:

- Các hành động thay đổi dữ liệu nguy hiểm (Write, Update, Delete)
- Gửi email trực tiếp cho khách hàng
- Khóa tài khoản người dùng

2. `interrupt_after`

Ngắt sau khi tạo nháp

Dừng đồ thị ngay sau khi node chạy xong để con người đánh giá kết quả.

Trường hợp áp dụng:

- Tạo báo cáo phân tích tổng hợp
- Sinh mã nguồn (Gen code) tự động
- Soạn thảo các email nháp (Draft generation)

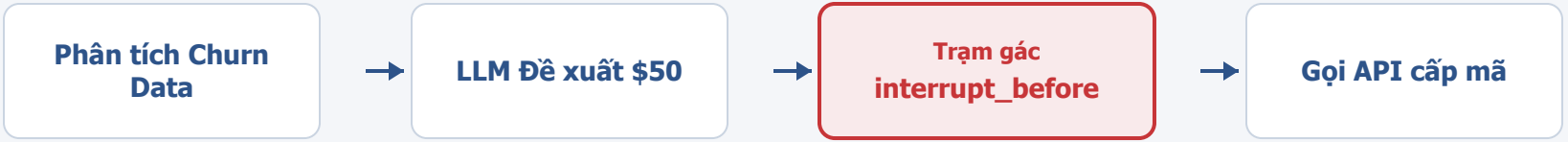
Kịch bản 1 - interrupt_before

Ví dụ: Phê duyệt ưu đãi giữ chân khách hàng (Churn Retention)

Tình huống: Agent dự đoán khách hàng có nguy cơ rời bỏ cao và đề xuất cấp mã giảm giá $50 để giữ chân.

Logic: Cấp mã là hành động tốn chi phí (Cost of Error → Mất tiền). Phải ngắt trước khi gọi API apply_discount.

Luồng xử lý (Flowchart)



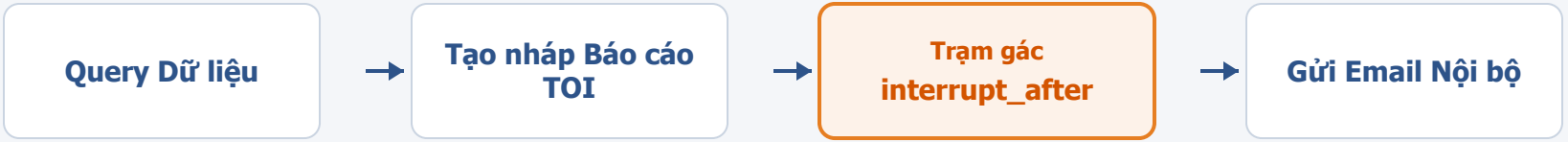
Kịch bản 2 - interrupt_after

Ví dụ: Review Báo cáo Total Operating Income (TOI)

Tình huống: Agent tổng hợp dữ liệu giao dịch và viết báo cáo phân tích TOI chi tiết cho phân khúc thẻ tín dụng.

Logic: Viết báo cáo không gây hại, gửi đi mới có rủi ro. Ta cho phép node viết chạy xong, rồi ngắt để duyệt nháp.

Luồng xử lý (Flowchart)



Vòng đời của một Graph bị Interrupt

1

Start

invoke(input, config)

2

Run

Đồ thị thực thi các node an toàn.

3

Suspend

Chạm trạm gác. Trạng thái lưu vào DB.

Code Python kết thúc tiến trình.

4

Review

Con người truy cập UI để đọc bản nháp qua **get_state**.

5

Resume

Gọi **update_state** và **invoke** để chạy tiếp.

Bước 1: Khai báo trạm kiểm duyệt trong Code

Khai báo Checkpointer:

Sử dụng **`MemorySaver()`** để lưu lại toàn bộ trạng thái (state) của đồ thị tại điểm dừng.

Thiết lập Trạm gác:

Khai báo danh sách các node nguy hiểm cần duyệt trước khi chạy qua tham số **`interrupt_before`**.

⚠️ *Ghi chú: Nếu quên truyền checkpointer, tham số interrupt_before sẽ bị lỗi.*

```
# Khởi tạo checkpointer

from langgraph.checkpoint.memory
import MemorySaver
memory = MemorySaver()

# Compile đồ thị, khai báo node nguy
hiểm

graph = builder.compile(
    checkpointer=memory,
    interrupt_before=[
        "apply_retention_offer"
    ]
)

# "apply_retention_offer" là tên node thực
thi hành động cấp ưu đãi
```

Bước 2: Chạy đồ thị và Kiểm tra trạng thái dừng

Chạy đồ thị với Thread ID:

Khai báo cấu hình định danh `thread_id`. Đồ thị sẽ tự động thực thi và dừng lại ngay trước node nguy hiểm đã được thiết lập.

Kiểm tra trạng thái dừng:

Hệ thống bên ngoài hoặc UI có thể truy vấn trạng thái hiện tại thông qua phương thức `get_state()`.

⚠️ *Kết quả trả về sẽ hiển thị rõ Node tiếp theo đang chờ được con người phê duyệt.*

```
# 1. Chạy đồ thị với Thread ID
config = {
    "configurable": {"thread_id": "churn_88"}
}
initial_input = {"customer_id": 88}
state = graph.invoke(initial_input, config)

# 2. Kiểm tra trạng thái dừng (UI)
curr_state = graph.get_state(config)
print("Chờ duyệt:", curr_state.next)

# Kết quả đầu ra:
# ('apply_retention_offer',)
```

Code Walkthrough - Phân tích Pending Action

Bước 3: Đọc dữ liệu Agent đề xuất (Đẩy lên UI)

```
# Lấy dữ liệu (state) Agent đã sinh ra trước khi ngắt
pending_data = current_state.values

print(f"Khách hàng: {pending_data['customer_id']}")
print(f"Đề xuất: {pending_data['proposed_action']}")
print(f"Lý do: {pending_data['reasoning']}")

# Dữ liệu này sẽ được render lên Streamlit UI
```



Giao diện UI Giả Lập

Customer ID
88

Proposed Action
Offer $50

Reasoning
Churn probability 85%

Approve

Bước 4: Cập nhật quyết định và Đánh thức đồ thị

Cập nhật State:

Gửi quyết định phê duyệt bằng cách thiết lập giá trị `human_approved` thành `True`.

Đánh thức đồ thị:

Gọi lại hàm thực thi đồ thị và truyền giá trị `None` để kích hoạt tiến trình tiếp tục chạy từ điểm dừng.

```
# Giả sử con người bấm "Approve" trên UI
# Cập nhật human_approved vào State
graph.update_state(
    config,
    {"human_approved": True}
)

# Đánh thức đồ thị chạy tiếp
final_state = graph.invoke(
    None, config
)

print("Quy trình hoàn tất!")
```

Edge Cases - Khi con người "bỏ đi ăn trưa"

Bài toán: Stale State / Timeout

⚠️ VẤN ĐỀ & RỦI RO (STALE STATE)

Agent đề xuất mua cổ phiếu lúc **10:00 AM** dựa trên giá hiện tại. Manager đi họp và đến **2:00 PM** mới bấm "Approve".

Hậu quả: Giá biến động mạnh, giao dịch bị lỗi hoặc gây tổn thất nặng do mất đi bối cảnh thời gian thực (Real-time context).

💡 NGUYÊN TẮC THIẾT KẾ

Các quyết định có yếu tố thời gian (**Time-sensitive**) tuyệt đối không thể chờ con người phê duyệt vô thời hạn. Cần cấu hình cơ chế **Timeout** tự động hủy hoặc cảnh báo.



Kịch bản rủi ro thực tế:

Khách hàng yêu cầu rút tiền lúc **8:00 AM**. Hệ thống treo chờ duyệt đến **5:00 PM** khiến giao dịch bị trì trệ và khách hàng giận dữ.

Triển khai TTL (Time-To-Live) cho Approval

Thiết lập "Thời hạn sử dụng" cho một Pending Action

TTL là gì?

Khoảng thời gian tối đa yêu cầu duyệt có hiệu lực (ví dụ: 15 phút, 24 giờ).

Cơ chế lưu trữ:

Khi Agent chạm điểm interrupt, ngoài ghi vào state, hệ thống lưu thêm trường `timestamp_generated` vào Audit DB.

Logic xử lý khi Resume:

1. Người dùng bấm **Approve** trên hệ thống.
2. Backend tiến hành kiểm tra điều kiện: `Current_Time - timestamp_generated < TTL`
3. **Nếu hợp lệ:** Cho phép **Resume** đồ thị chạy tiếp.
4. **Nếu quá hạn:** Đổi trạng thái thành **Expired** và yêu cầu luồng chạy lại từ đầu.

Xử lý State quá hạn (Expired State) trong LangGraph

Cơ chế kiểm soát TTL

- Đảm bảo tính toàn vẹn của state bằng cách kiểm tra thời gian thực thi tối đa cho phép.
- Tránh các quyết định lỗi thời khi manager phê duyệt muộn hoặc hệ thống bị trì hoãn.
- Tự động chuyển luồng sang trạng thái thất bại **FAILED_TIMEOUT** khi vượt quá ngưỡng quy định.

Ví dụ logic kiểm tra TTL trong node

```python
def apply_retention_offer(state: GraphState):
    time_elapsed = time.time() - state["generated_at"]
    if time_elapsed > 3600: # Quá 1 giờ
        return {"status": "FAILED_TIMEOUT"}
    if not state.get("human_approved"):
        return {"status": "REJECTED_BY_HUMAN"}
    # Thực thi logic thật nếu pass...
    return {"status": "SUCCESS"}
```

Handling Rejections - Xử lý khi bị từ chối

Chuyện gì xảy ra khi con người bấm "Reject"?

Định tuyến lại (Reroute) thay vì Abort

Khi `human_approved == False`, đồ thị không nhất thiết phải chết.

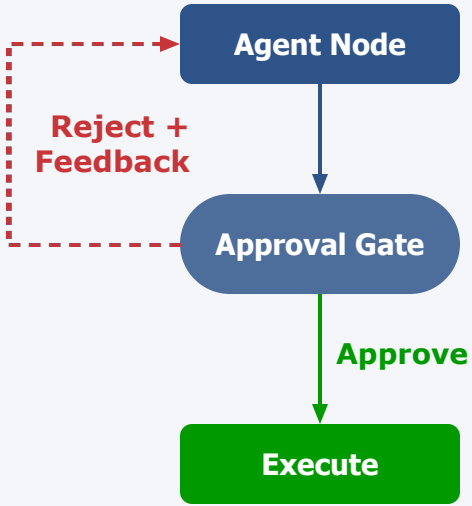
1. **Abort (Hủy hoàn toàn):**
   Kết thúc đồ thị và gửi thông báo cho User.

Reroute to LLM (Sửa sai):
Gửi phản hồi (Feedback) ngược lại node Agent.

2. *"Mức $50 quá cao, ngân sách TOI tháng này đã cạn. Đề xuất lại mức $20."*

Agent nhận context mới → Gen nháp mới → Chạm interrupt lần 2.

Feedback Loop Workflow



VINUNIVERSITY

UI/UX Design for HITL - Avoiding Alert Fatigue

Designing human interfaces that don't drive operators crazy.

5

Giao diện tồi = Quyết định tồi

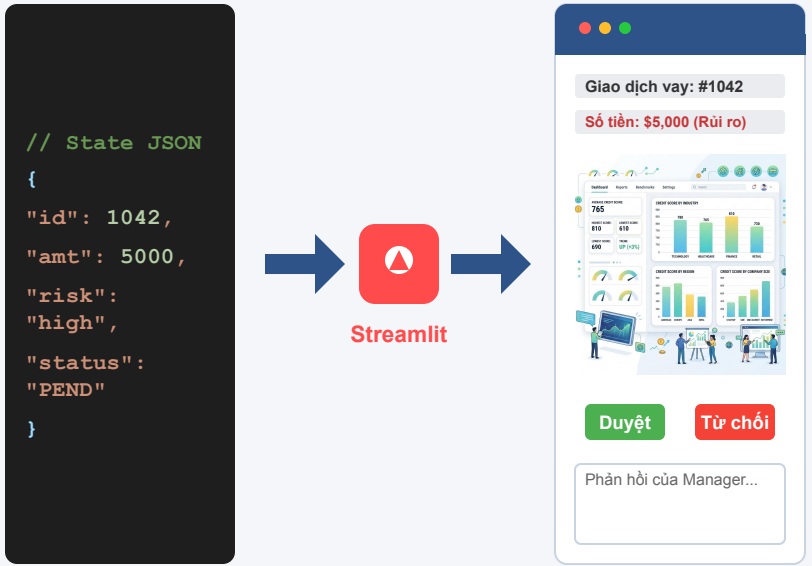
Thách thức & Giải pháp

Vấn đề: LangGraph checkpoint chỉ là dữ liệu JSON thô trong database. Con người không thể đọc JSON để phê duyệt nhanh chóng.

Mục tiêu HITL UI:
- Chuyển dữ liệu thô thành giao diện trực quan.
- Cung cấp đủ ngữ cảnh ra quyết định dưới **10 giây**.

Công cụ: Streamlit là lựa chọn tối ưu giúp dựng nhanh Data App bằng Python.

Luồng chuyển đổi dữ liệu trực quan



Cấu trúc chuẩn của một yêu cầu phê duyệt

Một Action Card tốt phải trả lời được **3 câu hỏi ngay lập tức**: *Ai? Cái gì? Mức độ tự tin?*

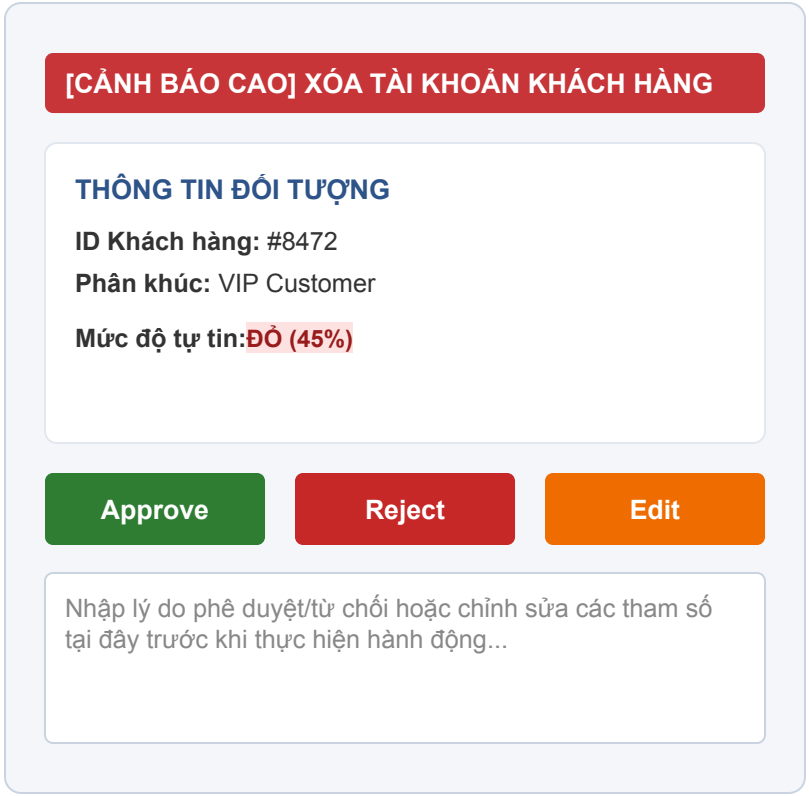
Header: Loại hành động cụ thể.
Ví dụ: [CẢNH BÁO CAO] - Xóa tài khoản

Target (Đối tượng): ID, Tên, Phân khúc.
Ví dụ: Customer 8472 - VIP

Agent Confidence: Hiển thị điểm tự tin rõ ràng kèm màu sắc chỉ báo (Xanh/Vàng/Đỏ).

Action Panel: 3 nút tương tác cơ bản:

- **Approve** (Chạy tiếp)
- **Reject** (Hủy bỏ)
- **Edit** (Sửa tham số trước khi chạy)



Sức mạnh của Diff View (Hiển thị thay đổi)

Đừng chỉ nói, hãy chỉ ra sự khác biệt (Delta)

Khái niệm Diff View

Lấy cảm hứng từ Git. UI highlight chính xác những trường dữ liệu thay đổi khi Approve.

Ví dụ thực tế

Agent đề xuất cập nhật trạng thái profile khách hàng từ hoạt động sang đóng băng.

Lợi ích vượt trội

Tránh tình trạng nhắm mắt duyệt bừa. Con người nhìn thấy ngay "hậu quả" của hành động.

profile_update_diff.git

```
// Git-style Delta Visualization
{
"user_id": 8472,
"name": "VIP Customer",
- "status": "Active"
+ "status": "Frozen"
"updated_by": "Agent_v2",
"confidence": 0.94
}
```

Explainable AI (XAI) trên Giao diện

Không nên bắt người dùng phải chơi trò đoán mò

Vấn đề của Black Box

Khi Agent chỉ đưa ra quyết định mà không giải thích:

"Tôi đề nghị đóng băng thẻ tín dụng này. Confidence: 90%." ➔ *Reviewer: "Tại sao??"*

Yêu cầu bắt buộc

UI phải hiển thị trực quan khối **Reasoning (Lập luận)** do mô hình tự động sinh ra để giải thích rõ ràng cho người duyệt.

LẬP LUẬN CỦA AGENT (REASONING)

Ví dụ chuẩn mực về giải thích:

Đề xuất khóa tài khoản vì các lý do sau:

- *Inactivity > 24 tháng*
- *Balance = $0*
- *Không có giao dịch phát sinh từ IP thường trú trong 6 tháng qua*

Traceability - Cung cấp bằng chứng (Evidence)

Tiết kiệm số lần click chuột (Click-saving Rule)

Nguyên tắc cốt lõi

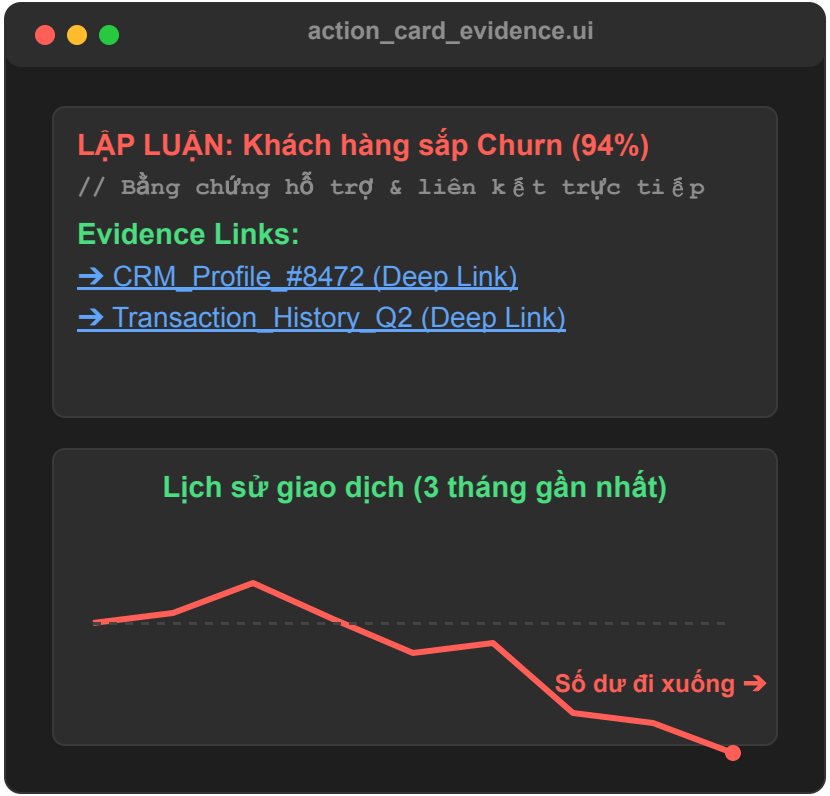
Nếu người duyệt phải mở tab mới, đăng nhập CRM và tự gõ ID khách hàng để đối soát → Quy trình sẽ bị chậm.

Tích hợp Deep Links

Đính kèm hyperlink trỏ thẳng đến Dashboard hoặc Lịch sử giao dịch của đối tượng đang xem xét.

Trực quan hóa Data Snippets

Hiển thị biểu đồ mini (Sparkline) về lịch sử giao dịch 3 tháng gần nhất ngay trên Action Card.



Kẻ thù số 1 - Alert Fatigue

Hội chứng Mệt mỏi vì Cảnh báo (Alert Fatigue)

Bài học từ Cybersecurity

Trong các trung tâm SOC (Security Operations Center):

Nếu chuyên gia nhận **500 cảnh báo** mỗi ngày, họ sẽ mất độ nhạy bén với các mối nguy hiểm thực sự.

➔ *Cảnh báo tràn ngập dẫn đến việc bỏ sót các sự cố nghiêm trọng.*

Rubber Stamping (Đóng dấu mù)

Khi số lượng interrupt quá lớn, con người sẽ hình thành phản xạ click **"Approve"** liên tục như một cỗ máy.

Lúc này, chốt chặn HITL hoàn toàn vô tác dụng.

An toàn hệ thống bị phá vỡ nhưng vẫn mang vỏ bọc **"đã được con người duyệt"**.

Tác động của Alert Fatigue đến Vận hành

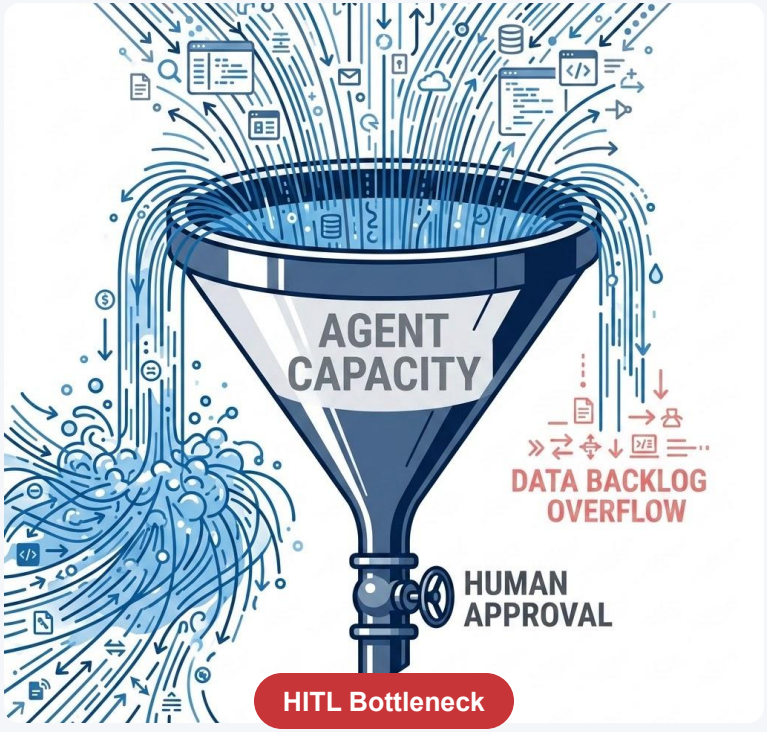
HITL trở thành Nút thắt cổ chai (Bottleneck)

Thách thức Vận hành

- **Tốc độ nghẽn:** Agent xử lý cực nhanh **10,000 req/phút**, trong khi con người (Human) chỉ duyệt được tối đa **5 req/phút**.
- **Chi phí tăng vọt:** Giữ con người trong vòng lặp liên tục đẩy chi phí vận hành Agent đắt đỏ hơn cả quy trình thủ công truyền thống.

Giải pháp Tối ưu

- **Giảm tải Interrupt:** Kết hợp lọc **Confidence Routing** chính xác và cơ chế duyệt hàng loạt **Batch Approvals** để giải phóng sức người.



Giải pháp: Batch Approvals (Phê duyệt hàng loạt)

Xử lý số lượng lớn bằng Bulk Actions

Khái niệm cốt lõi

Thay vì gửi **50 thông báo riêng lẻ** gây ngắt quãng và kiệt sức (Alert Fatigue)...

Hệ thống sẽ **gom tất cả lại thành một lô (Batch)** duy nhất để xử lý tập trung.

Quy trình vận hành (Workflow)

1. **Gom Action tương tự:** Agent tự động tổng hợp các tác vụ rủi ro Thấp/Trung bình vào một bảng dữ liệu.
2. **Review định kỳ:** Manager chỉ cần kiểm tra bảng tổng hợp **2 lần/ngày** (ví dụ: 10h sáng và 4h chiều).
3. **Duyệt nhanh:** Sử dụng giao diện Data Grid (dạng Excel), quét mắt nhanh và bấm **"Select All"** → **"Approve Selected"**.

Thiết kế Batch UI An toàn

Nguyên tắc vàng khi thiết kế Batch UI

1. Không trộn lẫn Rủi ro: Không bao giờ để lệnh **DROP TABLE** chung danh sách tick chọn với các lệnh Update_Email.

2. Sắp xếp thông minh: Đẩy các case có **Confidence thấp nhất** lên đầu bảng để người duyệt chú ý.

3. Nổi bật bất thường: Dùng màu cảnh báo nếu trong Batch có một action có **giá trị outlier** (Ví dụ: 1 lệnh $1,000 xen lẫn 49 lệnh $10).

Streamlit UI Mockup

[x]	Action	Conf.	Value
[x]	**Refund User_A**	**0.71**	**$1,000 (!!)**
[x]	Refund User_B	0.84	$10
[x]	Refund User_C	0.89	$10
[x]	Refund User_D	0.92	$10

Tóm tắt UI/UX Design cho HITL

Checklist thiết kế Giao diện Duyệt Agent

- **Action & Đối tượng:** Hiển thị rõ ràng, nhận biết được ngay trong 3 giây.
- **Diff View:** So sánh trực quan dữ liệu Trước và Sau khi Agent thay đổi.
- **Agent Reasoning:** Đưa ra lý do duyệt rõ ràng, loại bỏ hoàn toàn ảo giác.
- **Deep Links/Evidence:** Đính kèm nguồn dữ liệu gốc để tránh phải đổi Tab làm việc.
- **Batch Approval:** Hỗ trợ phê duyệt hàng loạt cho các tác vụ lặp đi lặp lại.

"Tôn trọng thời gian và giới hạn nhận thức (Cognitive Load) của người duyệt là chìa khóa để triển khai Agent thành công."

VINUNIVERSITY

Audit Trails, Compliance, & Continuous Learning

6

Enterprise readiness and using HITL data to improve the model.

Bức tranh Pháp lý & Tuân thủ (Compliance)

Audit Trail: Không chỉ là Log, mà là Sinh mệnh Pháp lý

- **Vấn đề:** Quyết định AI tác động đến tiền khách hàng (phân tích TOI, dự đoán Churn) bắt buộc phải giải trình được trước Ngân hàng Nhà nước hoặc tổ chức kiểm toán.
- **Tiêu chuẩn khắt khe:** SOC2, GDPR, ISO 27001 yêu cầu khả năng truy xuất nguồn gốc (Traceability) cho mọi hành động thay đổi dữ liệu (Write/Update).
- **Nguyên tắc Bất di bất dịch:** Audit Trail phải xây dựng từ **Ngày 1**. Không thể "gắn thêm" (retrofit) log hồi tố sau khi Agent đã lên Production và gây lỗi.

Hành lang bảo vệ Pháp lý cho Agent



Ghi lại mọi khoảnh khắc quyết định

```python
from pydantic import BaseModel
from datetime import datetime

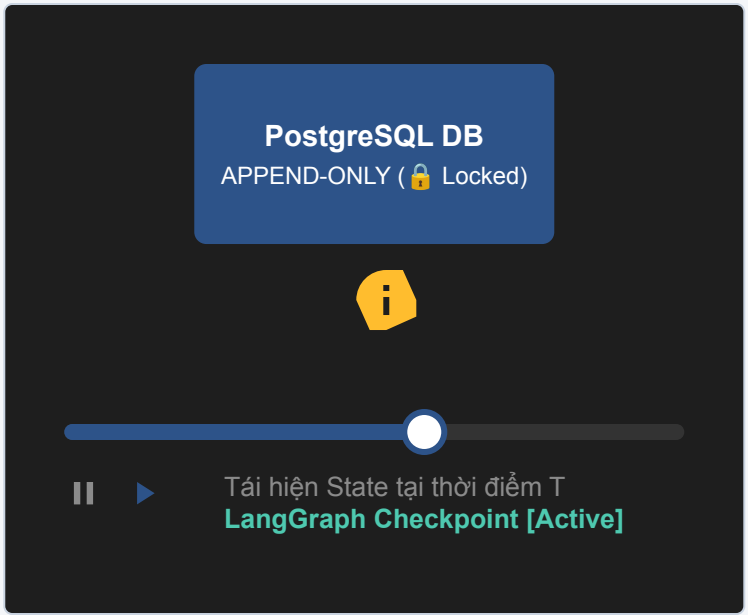
class AuditEntry(BaseModel):
    timestamp: datetime# Thời điểm chính xác
    thread_id: str# Định danh phiên (VD: churn_case_99)
    agent_id: str# Tên LLM/Agent thực thi
    action: str# Hành động (VD: apply_discount)
    confidence: float# Điểm tự tin ban đầu (0.0 - 1.0)
    risk_level: str# low / medium / high
    reviewer_id: str | None# ID của nhân sự duyệt (nếu có)
    decision: str# auto_executed / approved / rejected / edited
    reasoning: str | None# Lập luận của Agent & Lý do của Reviewer
    execution_time_ms: int# Thời gian xử lý
```

Append-only PostgreSQL & Session Replay

Tính Bất biến (Immutability) và Tái hiện (Replay)

- **Append-Only DB**
  - Cơ sở dữ liệu Audit (PostgreSQL) chỉ cho phép **THÊM (Insert)**, tuyệt đối không cho phép SỬA/XÓA (Update/Delete), kể cả quyền Admin. Bảo đảm tính toàn vẹn.
- **Session Replay (Tái hiện phiên):**
  - Kết hợp LangGraph Checkpointer, hệ thống cho phép **"tua lại" chính xác context** tại thời điểm T và màn hình UI lúc bấm "Approve".
  - Cực kỳ quan trọng để debug lỗi.

Mô phỏng cơ chế Replay & Immutable



Decision Analytics - Đo lường Sức khỏe Hệ thống

Agent của bạn thực sự đang làm tốt đến đâu?

Thay vì chỉ đo lường số lượng task, chúng ta phải đo lường "Chất lượng của sự cộng tác" giữa Người và AI thông qua 3 Metrics cốt lõi:

01. APPROVAL RATE

Tỷ lệ Phê duyệt

% Approve

Tỷ lệ phần trăm số lần con người thực hiện phê duyệt mà không cần bất kỳ chỉnh sửa nào.

02. RESPONSE TIME

Thời gian phản hồi

Avg. Time

Thời gian trung bình tính từ lúc Agent tạo bản nháp cho đến khi con người đưa ra quyết định.

03. OVERRIDE RATE

Tỷ lệ Bác bỏ/Sửa đổi

% Override

Tỷ lệ phần trăm số lần con người chủ động bấm Reject (bác bỏ) hoặc Edit (sửa đổi).

Phân tích Metrics (Insights from Data)

Đọc vị các chỉ số HITL

01. APPROVAL RATE > 95%

Tỷ lệ Phê duyệt Cao

- **Tín hiệu tốt:** Agent rất chính xác.
- **Hành động:** Nới lỏng Threshold (ví dụ: 0.85 → 0.80) để tăng tính tự động hóa.
- **Cảnh báo:** Người dùng bị Alert Fatigue & duyệt "nhắm mắt".

02. RESPONSE TIME QUÁ CAO

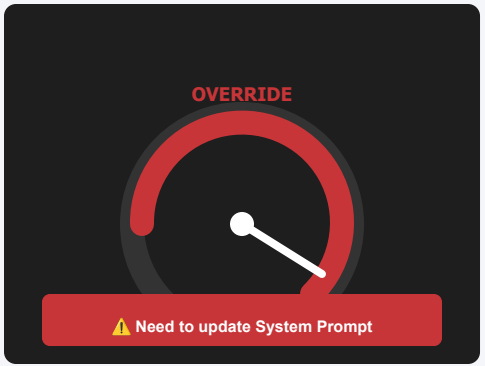
Phản hồi Chậm (> 4 giờ)

- **Nguyên nhân:** UI quá phức tạp, hoặc hệ thống gián đoạn công việc cốt lõi của user.
- **Hành động:** Chuyển sang cơ chế duyệt hàng loạt (Batch Approvals).

03. OVERRIDE RATE > 30%

Tỷ lệ Bác bỏ Cao

- **Nguyên nhân:** Prompt kém, Agent bị ảo giác, hoặc Threshold quá lỏng.



Progressive Autonomy (Tự trị Lũy tiến)

Xây dựng niềm tin theo thời gian (Trust Builds Incrementally)

Triết lý: Không ai tin tưởng một Agent vào ngày đầu tiên ra mắt. Hãy để AI chứng minh năng lực từng bước một.

THÁNG 1: HITL STRICT

Kiểm soát 100%

Con người thực hiện phê duyệt **100% các case** dự đoán Churn.

Thiết lập nền tảng an toàn tuyệt đối và thu thập dữ liệu vận hành ban đầu.

THÁNG 2: AUDIT & TUNING

Tự động hóa nhóm nhỏ

Nếu Approval Rate đạt **98%** ở nhóm "Hạn mức < $100" hạ threshold để **Auto-execute** nhóm này.

Bắt đầu nới lỏng kiểm soát dựa trên bằng chứng dữ liệu.

THÁNG 3: SCALE UP

Tối ưu hóa nguồn lực

Agent tự xử lý **60% workload** khối lượng thấp.

Con người giải phóng khỏi tác vụ lặp lại, chỉ tập trung vào nhóm khách hàng **Priority (VIP)**.

Vòng lặp Dữ liệu (The Data Flywheel)

HITL không phải là Thuế, nó là Khoản đầu tư Dữ liệu

01. HAI GÓC NHÌN VỀ CHI PHÍ HITL

Góc nhìn sai: Tốn chi phí xây dựng hệ thống, vẫn tốn thêm tiền vận hành cho con người phê duyệt.

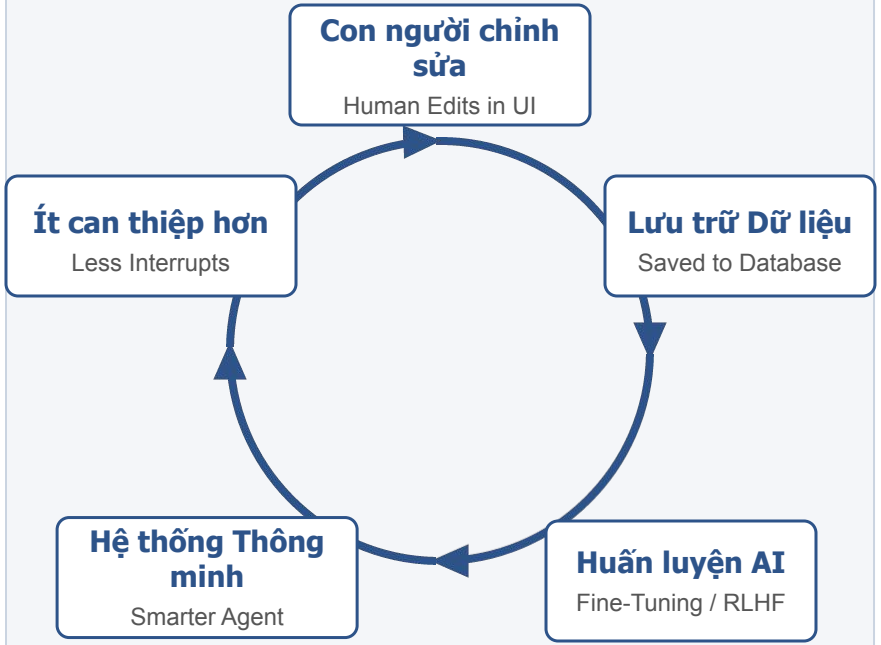
Góc nhìn đúng: Trạm kiểm duyệt (Approval Gate) là hệ thống thu thập dữ liệu huấn luyện miễn phí chất lượng cao.

02. KHÁI NIỆM VỀ FLYWHEEL

Tích lũy dữ liệu: Càng nhiều người dùng tương tác, hệ thống càng thu thập nhiều lượt chỉnh sửa và phản hồi (Edits/Rejects).

Tự động hóa tăng tiến: Kho dữ liệu huấn luyện lớn giúp tối ưu hóa Agent, giảm tần suất ngắt quãng và tiết kiệm chi phí.

03. SƠ ĐỒ VÒNG LẶP DỮ LIỆU KHÉP KÍN



Chuyển hóa Lỗi sai thành Preference Data

Từ nút "Edit" đến DPO (Direct Preference Optimization)

**Khi người dùng bấm Edit trên Streamlit và sửa đổi thông tin do Agent tạo ra,
chúng ta thu được cặp dữ liệu chuẩn xác để huấn luyện các mô hình theo chuẩn RLHF hoặc DPO:**

PROMPT (NGỮ CẢNH)

Đầu vào của hệ thống

Báo cáo lịch sử giao dịch của khách hàng.

REJECTED (OUTPUT LLM)

Thông tin bị từ chối

"Tặng voucher $50 vì khách hàng có nguy cơ rời bỏ".

CHOSEN (OUTPUT SỬA)

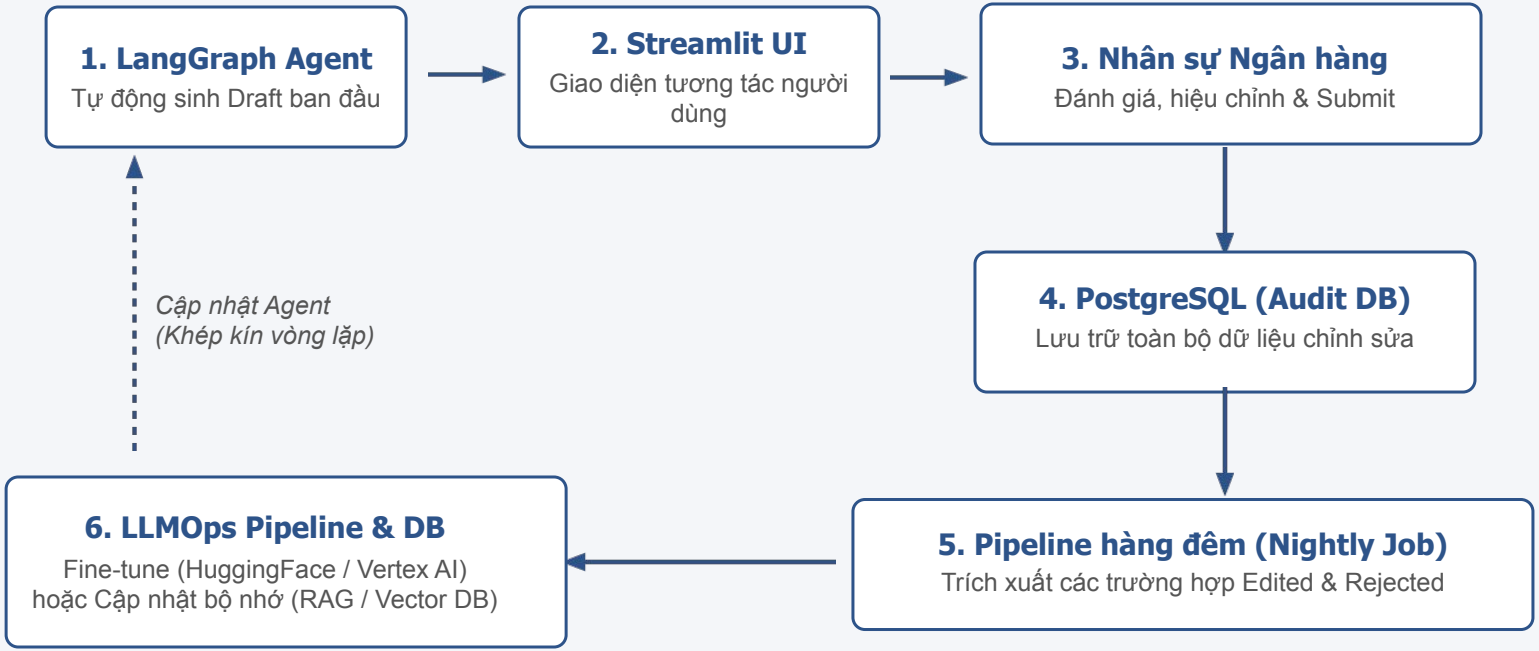
Con người hiệu chỉnh

"Không tặng voucher, chỉ gửi email chăm sóc vì lịch sử tín dụng xấu".

Kiến trúc Hệ thống Fine-Tuning tự động

Khép kín vòng lặp (Closing the Loop)

SƠ ĐỒ KIẾN TRÚC TỔNG THỂ (SYSTEM ARCHITECTURE DIAGRAM)



Tổng kết: Sẵn sàng cho Enterprise Production

BẮT BUỘC PHÁP LÝ

Audit Trail bắt buộc

Lưu giữ mọi thông tin về Agent ID, Confidence, và quyết định của người duyệt qua PostgreSQL append-only.

QUẢN TRỊ VẬN HÀNH

Tối ưu hóa hệ thống

Dùng Analytics (Approval Rate, Response Time) để hiệu chuẩn lại Threshold, chủ động ngăn chặn Alert Fatigue.

GIÁ TRỊ CỐT LÕI

Chuyển hóa dữ liệu

Biến sai sót của Agent thành Preference Data. Hành động Edit hôm nay chính là trí thông minh của Agent ngày mai.

Tổng kết — Key Takeaways

VINUNIVERSITY

Những ý chính cần nhớ sau buổi học hôm nay

1. HITL là **responsible deployment** — xây dựng trust incrementally, không phải dấu hiệu yếu kém
2. Confidence routing: **3 vùng** (auto / suggest / ask) + policy override cho high-risk ac-tions
3. Audit trail là compliance requirement — build from day one, **cannot retrofit** sau khi production
4. Progressive autonomy: start strict, đo metrics, nới dần — approval rate > 90% là tín hiệu tốt

Bức tranh toàn cảnh - System Architecture Diagram

Kiến trúc Hệ thống HITL Production-Ready

THÀNH PHẦN KIẾN TRÚC CỐT LÕI

Sự kết hợp giữa Orchestration, UI, và State Management:

1. AI Orchestration (LangGraph):

- Não bộ điều phối. Chứa các Node, Edge, và logic interrupt_before để tạm dừng chờ duyệt.

2. LLM Core (GPT-4 / Claude):

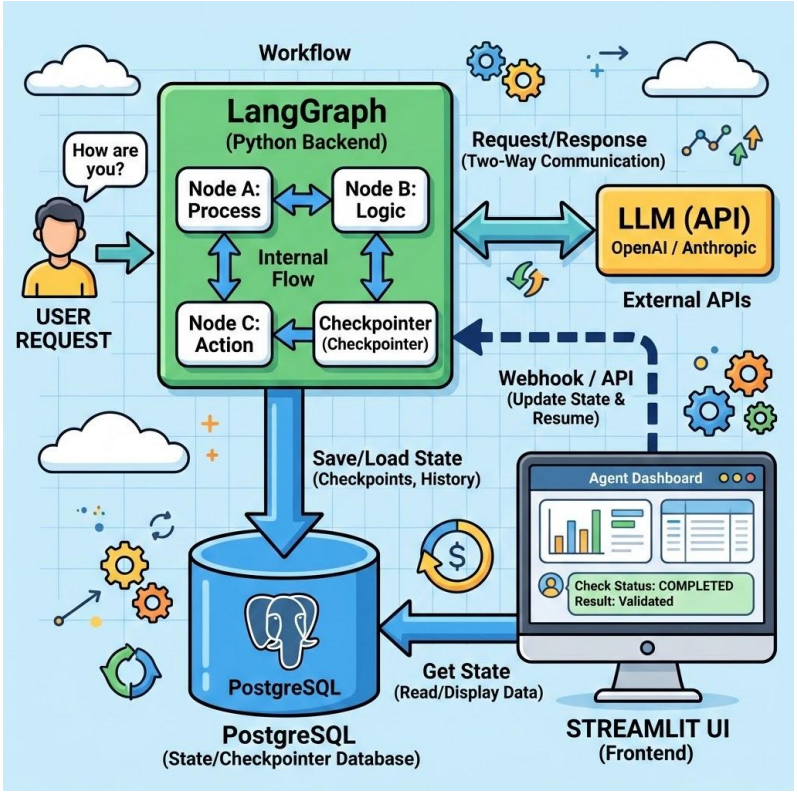
- Động cơ suy luận (Reasoning Engine). Đánh giá input và xuất ra Action kèm Confidence Score.

3. State & Audit (PostgreSQL):

- Đóng vai trò Checkpointer. Lưu trữ trạng thái Graph khi "ngủ đông" và ghi log Audit append-only.

4. Human Interface (Streamlit):

- Cổng giao tiếp. Lắng nghe State từ Postgres, render UI, và đẩy tín hiệu Resume ngược lại LangGraph.



Q&A



Thank you!

AICB-P2T3 · Ngày 27 · Human-in-the-Loop UX

github.com/vinuni-aicb

Liên hệ: instructor@vinuni.edu.vn